

MP4: Page Manager II

Phani Jyothi Kurada
UIN: 335006950
CSCE611: Operating System

Assigned Tasks

- **Part I: Support for Large Address Spaces** – Completed
- **Part II: Preparation of the PageTable Class to Manage Virtual Memory Pools** – Completed
- **Part III: Implementation of a Virtual Memory Allocator** – Completed

System Design

For this machine problem, the virtual memory pool manager was developed by building upon existing implementations from previous machine problems (MP2 and MP3). The development process is organized into multiple phases, each targeting specific objectives. Following the provided guidelines, the implementation is divided into three primary tasks:

- **Enhancing page table management** to support larger address spaces. This includes relocating directory and page table structures into the process memory pool.
- **Modifying the page table** to facilitate integration with the virtual memory system, enabling efficient memory access and allocation.
- **Implementing a simple virtual memory allocator** and connecting it with dynamic memory operations such as `new` and `delete`. The first 10 bits of an address are used to reference entries within the page directory.

Code Description

Files Modified: `page_table.H`, `page_table.C`, `vm_pool.H`, `vm_pool.C`, `cont_frame_pool.H` (from MP3), and `cont_frame_pool.C` (from MP3)

Part I: Recursive Page Table Lookup

- The main focus of this machine problem is the implementation of a recursive lookup mechanism for the page table, utilizing both the page directory and the page table.
- An offset is used to access the page directory or page table, as their contents represent frame numbers. This allows the CPU to treat these structures as standard memory pages.
- The address format for accessing the page directory is: `1023 | 1023 | X | Y | offset`. For accessing a page table page, the format is: `1023 | X | Y | offset`.
- In accordance with the MP4 instructions, the functions `PDE.address(unsigned long addr)` and `PTE.address(unsigned long addr)` have been implemented. These functions return pointers to the page directory entry (`*pde`) and page table entry (`*pte`), respectively.

- The `load()` method assigns the current page table instance and updates the CPU's CR3 register with the address of the page directory.
- The `enable_paging()` method sets the paging bit in the CR0 register, activating paging and enabling virtual memory management.

page_table.C: Constructor()

```

PageTable::PageTable()
{
    // Set initial base address for page table entries
    unsigned long start_address = 0;
    unsigned long total_frames = PageTable::shared_size / PAGE_SIZE; // Calculate required frames for kernel pool

    // Reserve memory for page table and directory
    unsigned long* page_table = (unsigned long*) (process_mem_pool->get_frames(1) * PAGE_SIZE);
    page_directory = (unsigned long*) (process_mem_pool->get_frames(1) * PAGE_SIZE);

    // Populate the page table
    for (int i = 0; i < total_frames; i++) {
        page_table[i] = start_address | 3; // Set present and writable bits
        start_address += PAGE_SIZE;
    }

    // Set up the page directory
    for (int i = 0; i < 1024; i++) {
        if (i == 0) {
            page_directory[i] = (unsigned long) (page_table) | 3;
        } else if (i == 1023) {
            page_directory[i] = (unsigned long) page_directory | 3;
        } else {
            page_directory[i] = 2; // Mark as not present
        }
    }

    start_address = 0;
    Console::puts("Constructed Page Table object\n");
}

```

Figure 1: Implementation of `constructor()`

page_table.C: pde_address()

```

unsigned long* PageTable::PDE_address(unsigned long addr) {
    unsigned long pde_addr = (addr >> 20); // Shift address by 20 bits to get PDE index
    pde_addr |= 0xFFFFF000; // Map PDE to right memory space
    pde_addr &= 0xFFFFF000; // Clear last 2 bits
    return (unsigned long*) pde_addr; // Address of PDE
}

```

Figure 2: Implementation of `pde_address()`

page_table.C: pte_address()

```

unsigned long* PageTable::PTE_address(unsigned long addr) {
    unsigned long pte_addr = (addr >> 10); // Shift address by 10 bits to get PTE index
    pte_addr |= 0xFFC00000; // Map PTE to right memory space
    pte_addr &= 0xFFC00000; // Clear last 2 bits
    return (unsigned long*) (pte_addr); // Address of PTE
}

```

Figure 3: Implementation of `pte_address()`

page_table.C: load()

```

void PageTable::load()
{
    current_page_table = this; // Assign the current instance to track the active page table
    write_cr3((unsigned long) (page_directory)); // Update CR3 register with page directory address
    Console::puts("Loaded page table\n");
}

```

Figure 4: Implementation of `load()`

page_table.C: enable_paging()

```
void PageTable::enable_paging()
{
    paging_enabled = true; // Mark paging as active
    write_cr0(read_cr0() | 0x80000000); // Set the paging bit in CR0 register to activate paging
    Console::puts("Enabled paging\n");
}
```

Figure 5: Implementation of enable_paging()

page_table.H:

```
class PageTable {
private:
    /* THESE MEMBERS ARE COMMON TO ENTIRE PAGING SUBSYSTEM */
    static PageTable * current_page_table; /* pointer to currently loaded page table object */
    static unsigned int paging_enabled; /* is paging turned on (i.e. are addresses logical)? */
    static ContFramePool * kernel_mem_pool; /* Frame pool for the kernel memory */
    static ContFramePool * process_mem_pool; /* Frame pool for the process memory */
    static unsigned long shared_size; /* size of shared address space */

    /* DATA FOR CURRENT PAGE TABLE */
    unsigned long * page_directory; /* where is page directory located? */

    static const int MAX_POOLS = 512; // Maximum number of pools allowed
    VMPool* registered_pools[MAX_POOLS]; // Fixed array to store pointers
    int pool_count = 0; // Tracks the number of registered pools

public:
    static const unsigned int PAGE_SIZE = Machine::PAGE_SIZE;
    /* in bytes */
    static const unsigned int ENTRIES_PER_PAGE = Machine::PT_ENTRIES_PER_PAGE;
    /* in entries */

    static void init_paging(ContFramePool * _kernel_mem_pool,
                           ContFramePool * _process_mem_pool,
                           const unsigned long _shared_size);
    /* Set the global parameters for the paging subsystem. */
}
```

Figure 6: Modifications in page_table.H

Part II: Enhancing Page Table for Virtual Memory Pools

- The PageTable class has been extended to include additional methods necessary for virtual memory management.
- The register_pool method maintains an array of all memory pools associated with the page table, while a separate variable tracks the total number of registered virtual memory pools. The free_page method releases a specific page in memory, identified by page_no.
- The page fault handler has been updated to validate memory accesses within the allocated virtual memory pools. Any access outside a valid region is flagged as an error.
- In case of an unexpected error, an error message is displayed and execution is halted. Furthermore, the page table now provides a free_page API, which takes a frame number as input and calls the release_frame API to deallocate the memory.

page_table.C: register_pool()

```

void PageTable::register_pool(VMPool * _vm_pool)
{
    if (pool_count == MAX_POOLS) {
        Console::puts("Virtual memory is full. Cannot register more pools!\n");
        assert(false);
    }
    registered_pools[pool_count++] = _vm_pool; // Add pool to array
    Console::puts("registered VM pool\n");
}

```

Figure 7: Implementation of `register_pool()`

`page_table.C: free_page()`

```

void PageTable::free_page(unsigned long _page_no) {
    unsigned long *pte = PTE_address(_page_no);
    if(*pte & 1) {
        process_mem_pool -> release_frames(*pte >> 12);
        *pte = 2;
    }
    write_cr3((unsigned long) page_directory); // Flush TLB
    Console::puts("freed page\n");
}

```

Figure 8: Implementation of `free_page()`

Part III: Implementing a Virtual Memory Allocator

- Additional APIs have been added to the `PageTable` class to support virtual memory management. The `register_pool` function maintains a list of all active virtual memory pools.
- All memory accesses are verified through the page fault handler. Any access outside the boundaries of a registered virtual memory pool is considered invalid.
- If an address fails validation, an error message is displayed and execution is halted. The page table also includes a `free_page` function, which calls the `release_frame` API with the corresponding frame number.
- **Constructor:** Initializes a `VMPool` object, associates it with the page table, and sets up memory regions using the base address and page size.
- **Allocate:** Reserves a memory block of a specified size, calculates the number of required pages, updates the memory map, and returns the starting virtual address of the allocated region.
- **Release:** Deallocates a memory block using its starting address, frees the associated physical pages, updates the memory segment list, and decrements the count of active allocations.
- **Is_legitimate:** Checks whether a given address belongs to an allocated memory segment by verifying its position relative to existing allocated regions. Returns `true` if the address is valid; otherwise, returns `false`.

vm_pool.C: Constructor()

```
VMPool::VMPool(unsigned long _base_address,
               unsigned long _size,
               ContFramePool *_frame_pool,
               PageTable *_page_table)
    :base_address(_base_address), size(_size), frame_pool(_frame_pool), page_table(_page_table) {
    page_table->register_pool(this);

    // Metadata (allocated and free region arrays) stored in first two pages
    allocated_regions = (Region*) base_address;
    free_regions = (Region*)(base_address + PageTable::PAGE_SIZE);

    // Initialize allocated region: metadata and free regions: entire pool (excluding metadata)
    num_allocated = 1;
    num_free = 1;
    allocated_regions[0].start_addr = base_address;
    allocated_regions[0].size = 2 * PageTable::PAGE_SIZE;           // 2 pages allocated for metadata
    free_regions[0].start_addr = base_address + 2 * PageTable::PAGE_SIZE;
    free_regions[0].size = size - 2 * PageTable::PAGE_SIZE;        // entire pool (excluding metadata)

    Console::puts("Constructed VMPool object.\n");
}
```

Figure 9: Implementation of constructor()

vm_pool.C: allocate()

```
unsigned long VMPool::allocate(unsigned long _size) {
    if (this == nullptr) {
        Console::puts("ERROR: `this` is NULL in VMPool::allocate()!\n");
        assert(false);
    }

    if(num_allocated >= MAX_REGIONS or num_free <= 0) {           // Check if memory regions are left
        Console::puts("Error in allocator, no memory\n");
        assert(false);
    }

    unsigned long pages_needed = (_size + PageTable::PAGE_SIZE - 1) / PageTable::PAGE_SIZE;
    unsigned long bytes_needed = pages_needed * PageTable::PAGE_SIZE;

    // Find a free region that can fit the allocation
    for (unsigned int i = 0; i < num_free; i++) {
        if (free_regions[i].size >= bytes_needed) {
            unsigned long allocated_addr = free_regions[i].start_addr;
            allocated_regions[num_allocated++] = {allocated_addr, bytes_needed};

            // Update free regions (split if remaining space)
            if (free_regions[i].size > bytes_needed) {
                free_regions[i].start_addr += bytes_needed;
                free_regions[i].size -= bytes_needed;
            } else {
                free_regions[i] = free_regions[num_free - 1];
                num_free--;
            }
            Console::puts("Allocated region of memory.\n");
            return allocated_addr;
        }
    }
    Console::puts("Allocation failed.\n");
    return 0;
}
```

Figure 10: Implementation of allocate()

vm_pool.C: release()

```
void VMpool::release(unsigned long _start_address) {
    // Find and remove from allocated regions
    for (unsigned int i = 0; i < num_allocated; i++) {
        if (allocated_regions[i].start_addr == _start_address) {
            free_regions[num_free++] = allocated_regions[i]; // Add to free regions
            allocated_regions[i] = allocated_regions[num_allocated - 1]; // Remove from allocated
            num_allocated--;
            // Free each page
            for (unsigned long addr = _start_address; addr < _start_address + allocated_regions[i].size; addr += PageTable::PAGE_SIZE) {
                page_table->free_page(addr);
            }
            break;
        }
    }
    Console::puts("Released region of memory.\n");
}
```

Figure 11: Implementation of release()

vm_pool.C: is_legitimate()

```
bool VMpool::is_legitimate(unsigned long _address) {
    Console::puts("Checking whether address is part of an allocated region.\n");
    if (_address == base_address) {
        return true;
    }
    // Check if address is in any allocated region
    for (unsigned int i = 0; i < num_allocated; i++) {
        unsigned long start = allocated_regions[i].start_addr;
        unsigned long end = start + allocated_regions[i].size;
        if (_address >= start && _address <= end) {
            return true;
        }
    }
    return false;
}
```

Figure 12: Implementation of is_legitimate()

page_table.C: handle_fault()

```
void PageTable::handle_fault(REGS * _r)
{
    Console::puts("Handling page fault...\n");
    unsigned long error_code = _r->err_code;

    // If the fault is caused by a reserved bit violation, ignore it
    if ((error_code & 0x1) == 1) {
        return;
    }

    if (!PageTable::current_page_table) {
        Console::puts("ERROR: No active page table! System halted.\n");
        assert(false); // Stop execution
    }

    if (!PageTable::current_page_table->pool_count) {
        Console::puts("ERROR: No registered VM pools! System halted.\n");
        assert(false);
    }

    unsigned long faulting_address = read_cr2();

    // Check if the memory address is inside a valid Virtual Memory Pool
    bool valid = false;
    for (int i = 0; i < PageTable::current_page_table->pool_count; i++) {
        if (!PageTable::current_page_table->registered_pools[i]) {
            Console::puts("ERROR: Found NULL pool in registered_pools!\n");
            assert(false);
        }
        if (PageTable::current_page_table->registered_pools[i]->is_legitimate(faulting_address)) {
            valid = true;
            break;
        }
    }

    // If the address is NOT in any valid pool - Segmentation Fault!
    if (!valid) {
        Console::puts("Segmentation Fault: Invalid Memory Access!\n");
        assert(false);
    }

    unsigned long* pde = PageTable::current_page_table->PDE_address(faulting_address);
    unsigned long* pte = PageTable::current_page_table->PTE_address(faulting_address);

    if (!(*pde & 1)) {
        *pde = (unsigned long) (PageTable::process_mem_pool->get_frames(1) * PAGE_SIZE) | 3;
    }

    if (!(*pte & 1)) {
        *pte = (unsigned long) (PageTable::process_mem_pool->get_frames(1) * PAGE_SIZE) | 3;
    }

    Console::puts("handled page fault\n");
}
```

Figure 13: Implementation of handle_fault()

vm_pool.H:

```

void PageTable::handle_fault(REGS * _r)
{
    Console::puts("Handling page fault...\n");
    unsigned long error_code = _r->err_code;

    // If the fault is caused by a reserved bit violation, ignore it
    if ((error_code & 0x1) == 1) {
        return;
    }

    if (!PageTable::current_page_table) {
        Console::puts("ERROR: No active page table! System halted.\n");
        assert(false); // Stop execution
    }

    if (!PageTable::current_page_table->pool_count) {
        Console::puts("ERROR: No registered VM pools! System halted.\n");
        assert(false);
    }

    unsigned long faulting_address = read_cr2();

    // Check if the memory address is inside a valid Virtual Memory Pool
    bool valid = false;
    for (int i = 0; i < PageTable::current_page_table->pool_count; i++) {
        if (!PageTable::current_page_table->registered_pools[i]) {
            Console::puts("ERROR: Found NULL pool in registered_pools!\n");
            assert(false);
        }
        if (PageTable::current_page_table->registered_pools[i]->is_legitimate(faulting_address)) {
            valid = true;
            break;
        }
    }

    // If the address is NOT in any valid pool - Segmentation Fault!
    if (!valid) {
        Console::puts("Segmentation Fault: Invalid Memory Access!\n");
        assert(false);
    }

    unsigned long* pde = PageTable::current_page_table->PDE_address(faulting_address);
    unsigned long* pte = PageTable::current_page_table->PTE_address(faulting_address);

    if (!(*pde & 1)) {
        *pde = (unsigned long) (PageTable::process_mem_pool->get_frames(1) * PAGE_SIZE) | 3;
    }

    if (!(*pte & 1)) {
        *pte = (unsigned long) (PageTable::process_mem_pool->get_frames(1) * PAGE_SIZE) | 3;
    }

    Console::puts("handled page fault\n");
}

```

Figure 14: Modifications in vm_pool.H

Testing

To compile and execute the test cases, the following commands were used:

```

make clean && make
make run

```

The testing was designed to validate the functionality of the page table and virtual memory management system. Specifically, we aimed to verify that:

- The `PageTable` class correctly handles memory allocation, deallocation, and page lookups.
- Virtual memory pools are properly registered, and memory accesses outside allocated regions are detected and flagged.

